

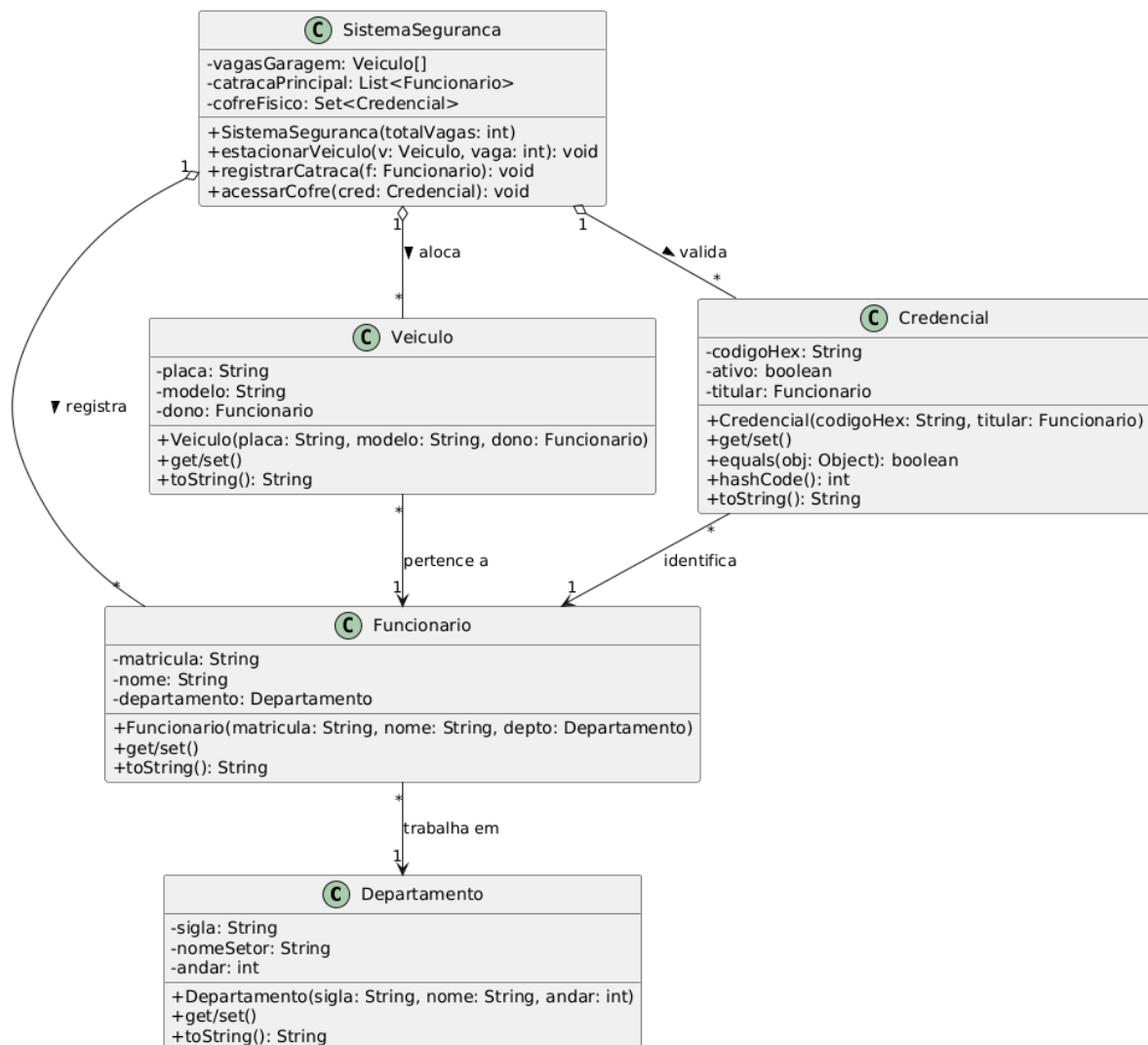
Operação "Zero Trust" (Domínio e Controle de Acesso)

O BRIEFING:

A *CyberCorp* sofreu uma falha de segurança terrível na semana passada. O sistema antigo permitiu que um invasor utilizasse um "crachá clonado" (um objeto diferente na memória, mas com o mesmo código hexadecimal) para acessar o cofre físico.

Você foi convocado como Engenheiro de Software Sênior para reconstruir o domínio da empresa do zero. Sua missão tem duas fases: primeiro, construir as entidades do negócio (Departamentos, Funcionários, Veículos); depois, blindar as portas de entrada usando o **Java Collections Framework**.

A Planta Arquitetural



O BACKLOG (Regras de Negócio)

Você deve codificar as classes obedecendo estritamente às regras abaixo:

RN01: A Base do Domínio (Departamento, Funcionario, Veiculo)

- Crie estas três classes primeiro. Aplique encapsulamento rigoroso (private para atributos, construtores preenchendo todos os dados e getters/setters).
- Não é necessário gerar equals() para elas, mas **todas** devem ter o método toString() sobrescrito para facilitar a leitura no console.

RN02: A Identidade Absoluta (Credencial)

- O sistema de segurança não olha para o rosto do funcionário, ele lê o chip do crachá. Duas credenciais são consideradas **idênticas** se, e somente se, o codigoHex for igual.
- **Ação:** Sobrescreva o equals() e o hashCode() **obrigatoriamente** usando o codigoHex como única chave matemática. (Deixe a sua IDE gerar isso para você evitar erros de digitação).

RN03: A Limitação Física (O Array da Garagem)

- Na classe SistemaSeguranca, a garagem de cimento tem tamanho fixo. O atributo vagasGaragem (Array) deve ser instanciado dentro do construtor, usando o parâmetro totalVagas.
- O método estacionarVeiculo deve alocar o objeto Veiculo na posição (índice) exata do Array e imprimir: *"Garagem: Veículo [Placa] estacionado na vaga [x]"*.

RN04: A Linha do Tempo (A List da Catraca)

- A recepção precisa de um histórico cronológico de quem passa. O atributo catracaPrincipal deve ser inicializado diretamente na declaração como um ArrayList<> para evitar *NullPointerException*.
- O método registrarCatraca deve adicionar o Funcionario na lista e imprimir: *"Catraca: Acesso liberado para [Nome]"*. **Importante:** A lista deve aceitar o mesmo funcionário múltiplas vezes no mesmo dia.

RN05: O Zero Trust (O Set do Cofre Físico)

- O cofre é a área de segurança máxima. **NUNCA** uma mesma credencial (ou seu clone) pode estar ativa lá dentro mais de uma vez simultaneamente. O atributo cofreFisico deve ser inicializado como um HashSet<>.
- O método acessarCofre deve tentar inserir a Credencial. Lembre-se: o método .add() de um Set retorna um boolean.
 - Se retornar true, imprima: *"Cofre: Acesso CONCEDIDO. Bem-vindo(a) [Nome do Titular]"*.
 - Se retornar false (tentativa de clonagem do código Hex), imprima um alerta grave: *"ALERTA MÁXIMO: Credencial [Hex] bloqueada! Tentativa de clonagem detectada."*

RN06: O Campo de Provas (MainTeste)

Crie uma classe com o método main para provar que a sua arquitetura resiste a um ataque:

1. Instancie **1 Departamento**, **1 Funcionário** e **1 Veículo**.
2. Instancie o SistemaSeguranca com apenas **2 vagas** de garagem.
3. Crie a credencial original: `c1 = new Credencial("FFF-999", true, func);`
4. Crie o CLONE (mesmo código, outro objeto na memória): `clone = new Credencial("FFF-999", true, func);`
5. Teste a Catraca: Passe o funcionário duas vezes. O console deve permitir ambos.
6. Teste o Cofre: Tente passar a `c1` e depois a credencial clone. O seu HashSet **deve** bloquear o clone e disparar o "ALERTA MÁXIMO".
7. Teste a Garagem: Estacione o carro na vaga 0. Depois, force um erro tentando estacionar na vaga 5. O sistema deve "capotar" com o erro vermelho `ArrayIndexOutOfBoundsException`, provando a rigidez dos Arrays abordada em aula.